

Advanced Process Mining — Assignment Part II

Group $\langle \text{GroupID} \rangle$
 $\langle \text{Member 1} \rangle, \langle \text{Member 2} \rangle, \langle \text{Member 3} \rangle$

July 5, 2026

Contents

1	Question 3: Trace Probabilities in GSLPNs	2
2	Question 5: Object-Centric Process Mining	3
2.1	(b) Conformance checking for Cameron	6

1 Question 3: Trace Probabilities in GSLPNs

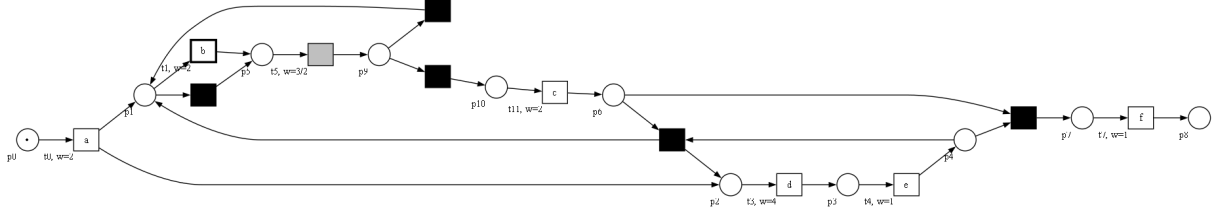


Figure 1: The GSLPN from the assignment (p.6), redrawn from the encoding used by `q3_gslpn.py`. Layout differs from the original figure (auto-generated left-to-right), but the places, transitions, weights and arcs are identical. Legend: plain box = timed, thick-bordered box = immediate (visible), grey box = τ and timed, black box = τ and immediate.

(a) Stochastic path with highest probability

Computed by `q3_gslpn.py`: every enabled transition set is split into immediate/timed per the priority rule, the winning class races by weight, and the highest-probability firing sequence projecting onto σ is found with a Dijkstra variant that maximizes a product of probabilities instead of minimizing a sum of costs.

Step	Trace so far	Marking	Transition fired	$P(\text{step})$
0	$\langle \rangle$	$M_0 = \{p_0\}$	—	—
1	$\langle a \rangle$	$M_1 = \{p_1, p_2\}$	t_0	1
2	$\langle a \rangle$	$M_2 = \{p_2, p_5\}$	t_2	$\frac{3}{5}$
3	$\langle a, d \rangle$	$M_3 = \{p_3, p_5\}$	t_3	$\frac{8}{11}$
4	$\langle a, d \rangle$	$M_4 = \{p_3, p_9\}$	t_5	$\frac{3}{5}$
5	$\langle a, d \rangle$	$M_5 = \{p_3, p_{10}\}$	t_{10}	$\frac{1}{4}$
6	$\langle a, d, c \rangle$	$M_6 = \{p_3, p_6\}$	t_{11}	$\frac{2}{3}$
7	$\langle a, d, c, e \rangle$	$M_7 = \{p_4, p_6\}$	t_4	1
8	$\langle a, d, c, e \rangle$	$M_8 = \{p_7\}$	t_6	$\frac{1}{3}$
9	$\langle a, d, c, e, f \rangle$	$M_9 = \{p_8\}$	t_7	1

Table 1: Path π_1 : $\langle a, d, c, e, f \rangle$ via $(t_0, t_2, t_3, t_5, t_{10}, t_{11}, t_4, t_6, t_7)$

$$P(\pi_1) = 1 \cdot \frac{3}{5} \cdot \frac{8}{11} \cdot \frac{3}{5} \cdot \frac{1}{4} \cdot \frac{2}{3} \cdot 1 \cdot \frac{1}{3} \cdot 1 = \frac{4}{275} \approx 0.0145$$

Step 2 is a race between t_1 (b , $w = 2$) and t_2 (τ , $w = 3$); firing t_1 would introduce a b that σ does not contain, so only t_2 can extend a match, at probability $\frac{3}{5}$. At every point where p_9 has a token (step 4→5), the alternative is the silent loop t_9 ($w = 3$) back to p_1 : since every extra loop multiplies the remaining probability by a factor strictly less than 1 (and re-visits p_1 without ever adding to the match, since t_1/b still cannot fire), looping can never exceed the direct continuation through t_{10} . The same argument rules out t_8 ($w = 2$) at step 8: taking it would force a second, unmatched $d/e/c$ before any further progress is possible. Hence π_1 above is the unique candidate and thus the maximum: $P(\pi_1) = \frac{4}{275} \approx 1.45\%$.

(b) Probability of observing trace σ

Unlike (a), π_1 is not the only path projecting onto σ : the silent loop $t_9 \rightarrow p_1 \rightarrow t_2 \rightarrow p_5 \rightarrow t_5 \rightarrow p_9$ can repeat any number of times (always avoiding t_1/b) before finally taking t_{10} , and its position can interleave differently with the d/e branch – both without changing the visible trace. This is

an infinite family of paths, so $P(\sigma)$ is computed (not enumerated) as the absorption probability, into the accepting state, of the embedded DTMC restricted to σ -consistent transitions – i.e. the solution of the linear system $x_s = \sum_{\text{edges } s \rightarrow s'} p \cdot x_{s'}$ over the (finite, 17-state) reachable marking graph, with $x_{\text{accept}} = 1$:

$$P(\sigma) = \frac{1252}{42267} \approx 0.0296 \text{ (2.96\%)}.$$

As expected, $P(\sigma) > P(\pi_1)$: it is the sum of π_1 's probability plus all its loop-extended variants. This was cross-checked by summing the same paths directly, truncated at an increasing maximum loop count – the partial sums converge to 1252/42267 (matching to 6 decimal places by 10 loop iterations), confirming the linear solve.

2 Question 5: Object-Centric Process Mining

The object-centric event log L records a collaborative coding process in a Git-based version control system. The object types are $OT = \{\text{branches, users, files, file versions}\}$ and the event types are $ET = \{\text{commit, pull, merge, push}\}$. File versions are linked to their parent file via the object-object relation `is_a`. When users commit, event-object qualifiers `produce` (new version) and `consume` (old version) record which file versions were created or superseded. The full log is provided as an OCEL 2.0 file `version_control.sqlite`.

(a) Checking C1, C2, C3 with OCPQ (4.5 points)

For each constraint we describe the OCPQ query used to check it against the event log L , show a screenshot of the query and its result, and briefly explain the underlying logic.

C1: “You should pull before you push, and you should not do anything between the pull and the push except for possibly a merge.”

Query. We use a nested query tree with three levels.

- **Root node** – all push events on a branch: variables e_2 : push, o_1 : branches; filter `E2O( $e_2, o_1$ )`.
- **Child node** – candidate pulls preceding this push on the same branch: variable e_1 : pull (inherits o_1); filters `E2O( $e_1, o_1$ )` and `e1.time() < e2.time()`.
- **Grandchild node** – events strictly between e_1 and e_2 , excluding merges: variable e_3 : *; filters `e1.id() != e3.id() && e2.id() != e3.id(), e1.time() < e3.time(), e3.time() < e2.time(), e3.attr("ocel:activity") != "merge"`.

Constraints: on the *child* (pull) node we require $|\text{grandchild}| \leq 0$, which keeps only the *clean* pull witnesses – pulls with nothing (other than possibly a merge) between them and the push. On the *root* (push) node we require $|\text{child}| \geq 1$: a push is compliant iff at least one clean witness pull remains.

Result and explanation. The query flags 4 of the 30 pushes as violations (see Figure 2). A push is only counted as compliant if it has an *immediately* preceding pull on the same branch, with no event other than a merge in between; existence of some *earlier*, non-adjacent pull is not sufficient, since anything (including another push or commit) sitting between that pull and the push disqualifies it as a witness. The four flagged pushes either have no preceding pull on their branch at all, or every earlier pull is separated from them by an intervening non-merge event.

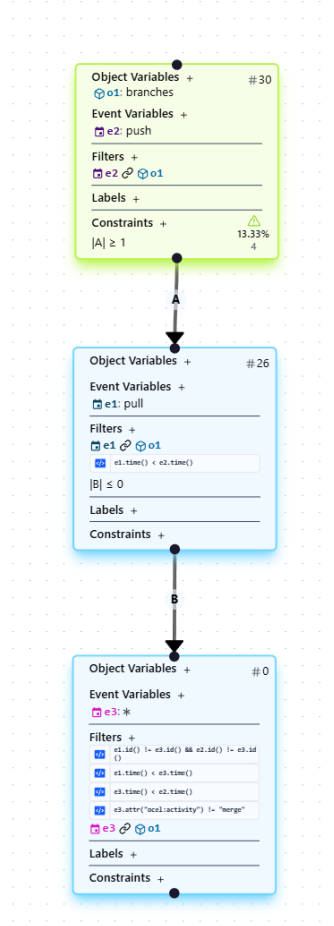


Figure 2: OCPQ query for C1 (three nested nodes) and result: 4 of 30 pushes violate C1 (86.67% compliance).

C2: “Do not alternate between branches: after committing to one branch b , you should not touch a different branch until having pushed to b .”

Query. Again a three-level nested query tree.

- **Root node** – all commits: variables e_1 : commit, o_1 : branches, o_2 : users; filters $E2O(e_1, o_1)$, $E2O(e_1, o_2)$.
- **Child node** – candidate later pushes to the same branch by the same user: variable e_2 : push (inherits o_1, o_2); filters $E2O(e_2, o_1)$, $E2O(e_2, o_2)$, $e1.time() < e2.time()$.
- **Grandchild node** – events by the same user on a *different* branch, strictly between e_1 and this candidate push: variable e_3 : *, o_3 : branches; filters $E2O(e_3, o_2)$, $E2O(e_3, o_3)$, $o3.id() != o1.id()$, $e1.time() < e3.time()$, $e3.time() < e2.time()$.

Constraints: on the *child* (push) node we require $|grandchild| \leq 0$, keeping only pushes to b that are not preceded by activity of the same user on a different branch. On the *root* (commit) node we require $|child| \geq 1$: a commit is compliant iff at least one such clean push exists afterwards.

Result and explanation. 2 of the 39 commits violate C2 (see Figure 3). A commit is compliant if the committing user eventually pushes to the same branch again without first touching a different branch; committing again, or pulling, on the *same* branch b in between does

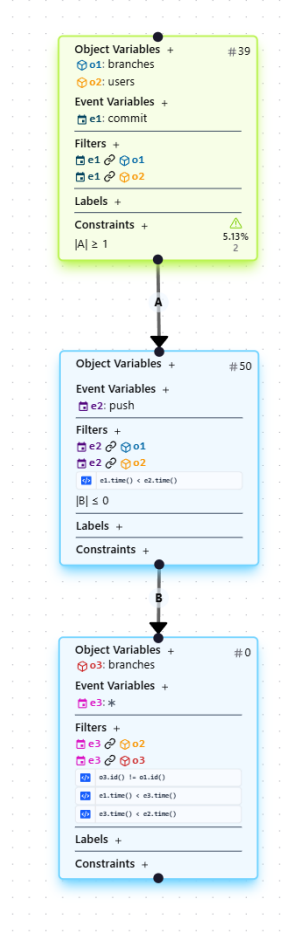


Figure 3: OCPQ query for C2 (three nested nodes) and result: 2 of 39 commits violate C2 (94.87% compliance).

not count against the constraint, only activity tied to a *different* branch object does. Of the two violating commits, at least one case corresponds to the user touching a different branch before ever pushing to b ; the other case may correspond to the user never pushing to b again at all – both are genuine violations of C2, but for different underlying reasons.

C3: “Avoid coordination problems: a coordination problem between two users $u_1 \neq u_2$ is a merge event that consumes two versions of the same file, where one version was committed by u_1 and the other by u_2 .”

Query. Unlike C1 and C2, C3 is not an existential (“does a witness exist”) condition but a direct pattern match, so a single flat query node suffices.

- Object variables: o_1, o_2 : file versions, o_3 : files, o_4, o_5 : users.
- Event variables: e_1 : merge, e_2, e_3 : commit.
- Filters: $E2O(e_1, o_1)@consume$, $E2O(e_1, o_2)@consume$, $O2O(o_1, o_3)@is_a$, $O2O(o_2, o_3)@is_a$, $E2O(e_2, o_1)@produce$, $E2O(e_3, o_2)@produce$, $E2O(e_2, o_4)$, $E2O(e_3, o_5)$; CEL constraints $o1.id() < o2.id()$ and $o4.id() != o5.id()$.

The condition $o1.id() < o2.id()$ imposes a canonical ordering on the two conflicting versions so that each real coordination problem is counted exactly once (without it, every coordination problem would be found twice, as (o_1, o_2) and (o_2, o_1)). The condition $o4.id() != o5.id()$ is the actual C3 condition: the two versions were committed by different users.

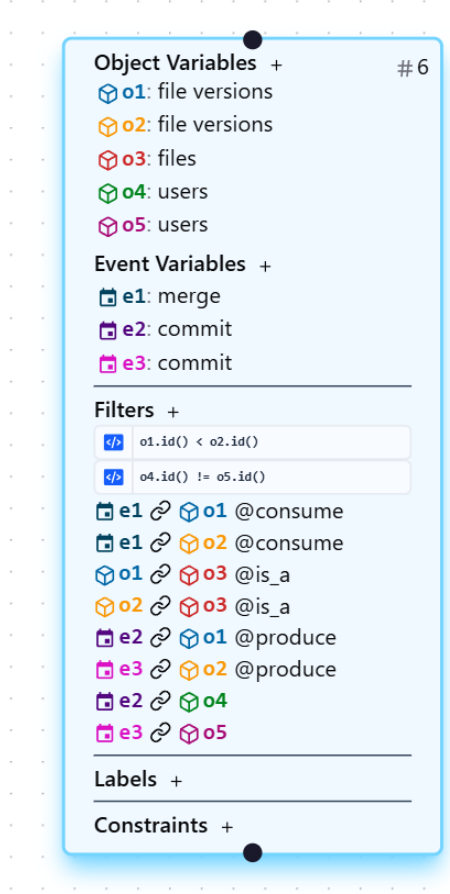


Figure 4: OCPQ query for C3 (single flat node) and result: 6 coordination problems found.

Result and explanation. The query returns 6 output bindings (see Figure 4), i.e. 6 coordination problems: pairs of file versions of the same file, produced by two different users, that were both consumed by the same merge event. Each binding directly witnesses one instance of the pattern described by C3, so no separate compliance percentage is computed here – the count itself is the answer. Note that this counts *version pairs*; if a single merge consumed more than two conflicting versions it would contribute multiple pairs to this count rather than being counted once per merge.

2.1 (b) Conformance checking for Cameron

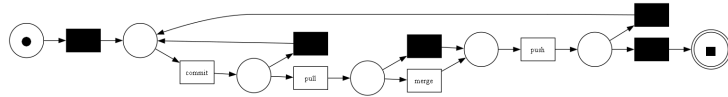


Figure 5: Per-object-type subnet of N_1 (identical for `branches` and `users`): $\text{commit}^+ \rightarrow \text{pull} \rightarrow [\text{merge}]? \rightarrow \text{push}$, looped. Generated by `q5b_conformance.py` (`build_n1_subnet`).

Applying the given definition of L_1^{Cameron} for $u = \text{Cameron}$ (`q5b_conformance.py`) yields 13 events, flattened below per object type.

Flattened traces

Table 2: Traces per branch (**branches** flattening of L_1^{Cameron}).

Branch	Trace
b1	commit → pull → push → commit → push → commit → commit → commit → pull → push
b3	commit → pull → push

Table 3: Trace for *Cameron* (**users** flattening of L_1^{Cameron}).

User	Trace
Cameron	commit → pull → push → commit → push → commit → commit → commit → pull → push → commit → pull → push

Token-based replay fitness

Computed with `pm4py.conformance_diagnostics_token_based_replay` against fig. 5.

Table 4: Token replay diagnostics — **branches** subnet.

Trace (branch)	Fit?	Fitness	Consumed	Produced	Missing	Remaining
b1	No	0.9474	19	19	1	1
b3	Yes	1.0000	7	7	0	0
Average		0.9737				

Table 5: Token replay diagnostics — **users** subnet.

Trace (user)	Fit?	Fitness	Consumed	Produced	Missing	Remaining
Cameron	No	0.9583	24	24	1	1
Average		0.9583				

$$\bar{f} = \frac{f_{\text{branches}} + f_{\text{users}}}{2} = \frac{0.9737 + 0.9583}{2} = 0.9660.$$

Alignments

Computed with `pm4py.conformance_diagnostics_alignments`. A plain activity is a synchronous move, **tau** a silent model move, and (**log:X**) a log-only move (deviation).

- b1 (fitness 0.9231): tau commit pull tau push tau commit tau (**log:push**) commit tau commit tau commit pull tau push tau
- b3 (fitness 1.0000): tau commit pull tau push tau
- Cameron / users (fitness 0.9375): tau commit pull tau push tau commit tau (**log:push**) commit tau commit tau commit pull tau push tau commit pull tau push tau

Average alignment fitness: 0.9615 (**branches**), 0.9375 (**users**). Both non-fit traces share the same single (**log:push**) deviation, which is the source of the missing/remaining token in tables 4 and 5 (used for C1 below).

Alignment-based precision (prefix automata)

Reconstructed via the Align-ET Conformance algorithm (Adriansyah et al., 2015): one node per log prefix, solid edges for activities the log continues with, dashed red edges for activities N_1 would additionally allow but that never occur (escaping edges, which reduce precision).

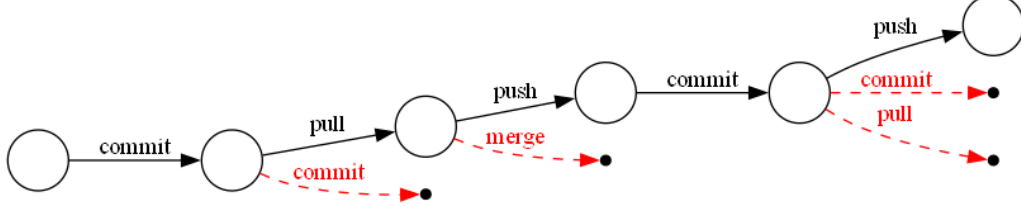


Figure 6: Prefix automaton — **branches** subnet.

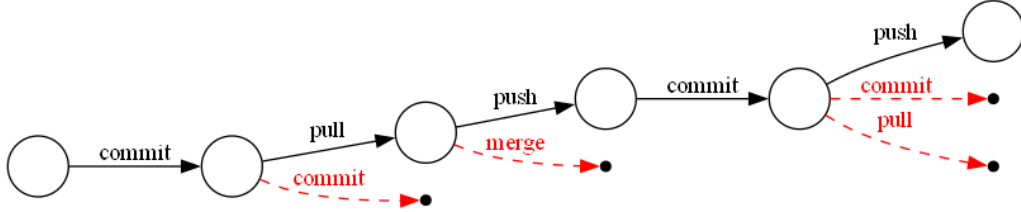


Figure 7: Prefix automaton — **users** subnet.

$$\bar{p} = \frac{p_{\text{branches}} + p_{\text{users}}}{2} = \frac{0.5385 + 0.5000}{2} = 0.5192.$$

Fitness and precision summary

Table 6: Aggregated conformance results for L_1^{Cameron} vs. N_1 .

Subnet	Token-replay fitness	Alignment precision
branches	0.9737	0.5385
users	0.9583	0.5000
Average	0.9660	0.5192

Checking C1 and C2 for Cameron

C1 (pull before push). Yes. N_1 encodes C1 directly (**push** is only enabled after a **pull** of the current round), so the single deviation found in both subnets *is* a C1 violation: the alignment pinpoints it as the **b1** push at event **e82**, occurring right after a commit with no intervening pull – matching the violation independently found in Q5a(a).

C2 (no branch-switching after commit). No. N_1 's **branches** subnet checks each branch in isolation and the **users** subnet discards branch identity, so neither can tell that an event belongs to a *different* branch than the one currently active – exactly what C2 requires. Indeed, *Cameron* committed to **b3** (event **e89**) while **b1** was still unpushed, a genuine C2 violation, yet it causes no missing/remaining token in either subnet; detecting it needs a query that keeps both object types in scope simultaneously, as in Q5a(a).